

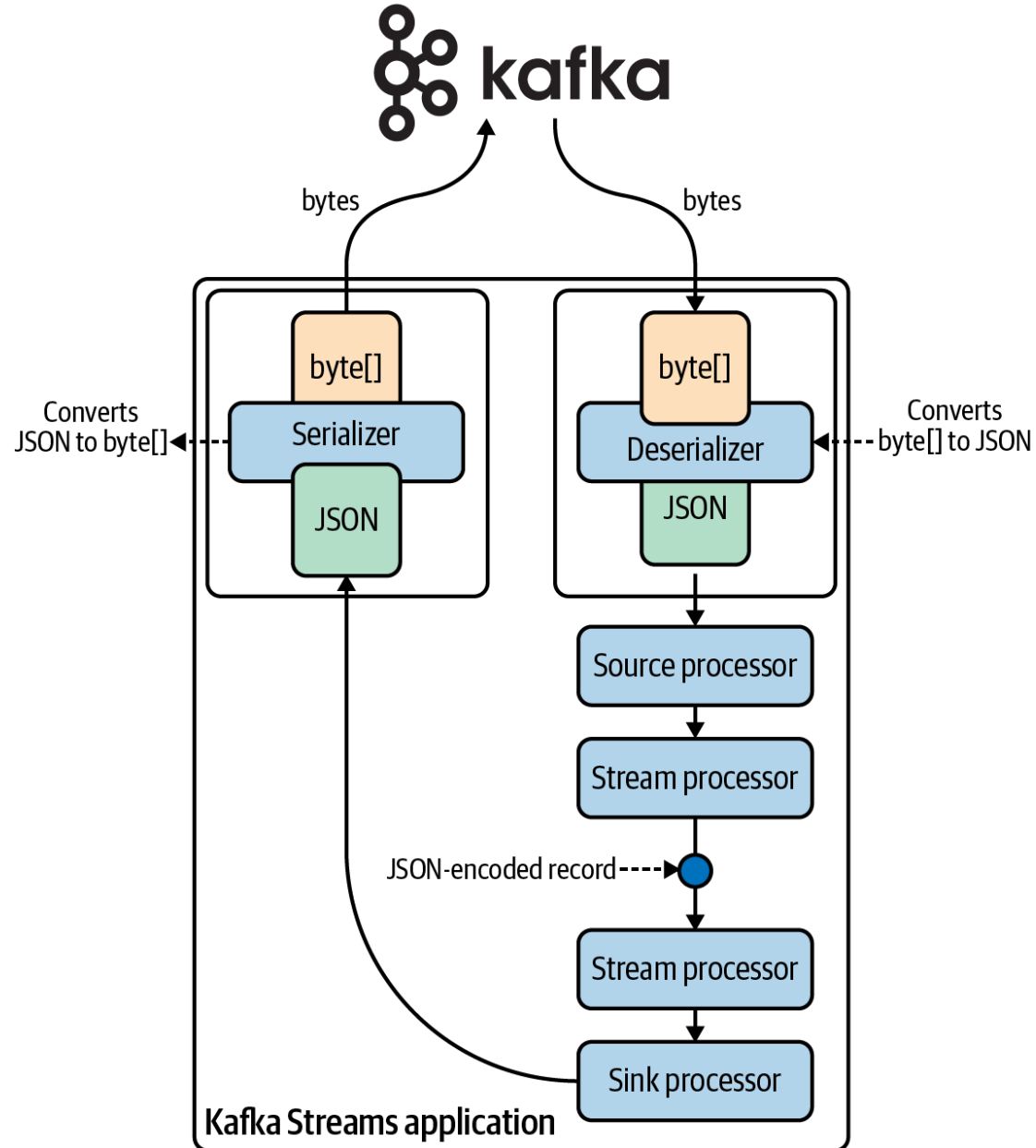
Kafka Producer - Java

Objectives Create Producer

- ❖ Create simple example that creates a ***Kafka Producer***
- ❖ Create a new replicated ***Kafka topic***
- ❖ ***Create Producer*** that uses topic to send records
- ❖ ***Send records*** with ***Kafka Producer***
- ❖ ***Send records asynchronously.***
- ❖ ***Send records synchronously***

Ser/Deserialization Architecture

Tos

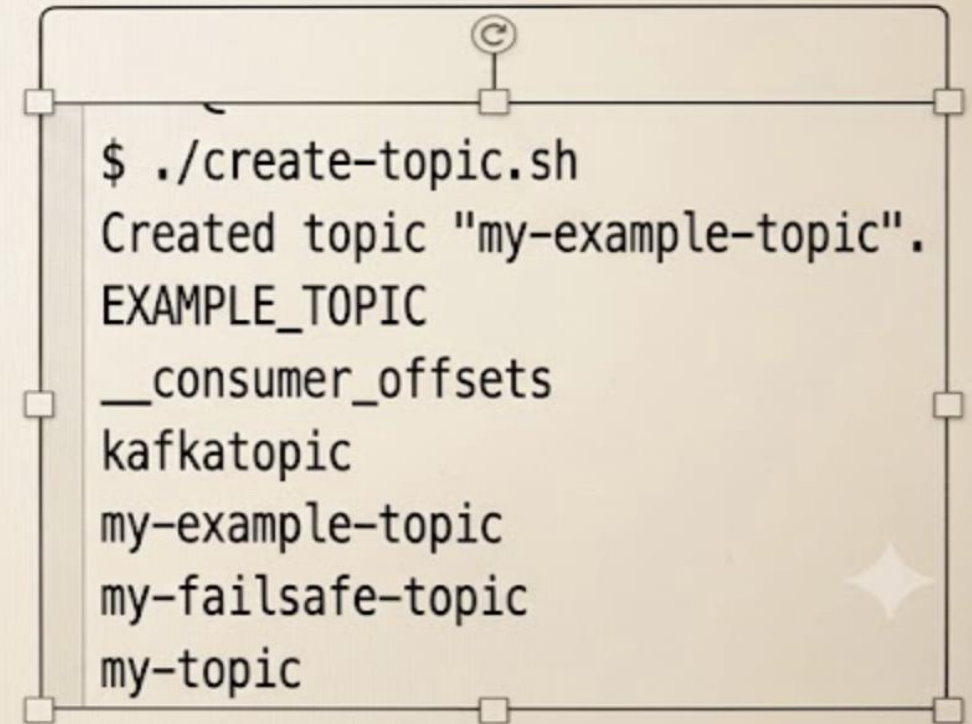


Create Replicated Kafka Topic

```
#!/usr/bin/env bash
cd ~/kafka-training

## Create topics
kafka/bin/kafka-topics.sh --create \
  --replication-factor 3 \
  --partitions 13 \
  --topic my-example-topic \
  --bootstrap-server localhost:9092

## List created topics
kafka/bin/kafka-topics.sh --list \
  --bootstrap-server localhost:9092
```



Gradle/Maven Build script

kafka-training x

```
1  group 'cloudurable-kafka'
2  version '1.0-SNAPSHOT'
3
4  apply plugin: 'java'
5
6  sourceCompatibility = 1.8
7
8  repositories {
9      mavenCentral()
10 }
11
12 dependencies {
13     compile 'org.apache.kafka:kafka-clients:0.10.2.0'
14     compile 'ch.qos.logback:logback-classic:1.2.2'
15 }
```

```
<dependency>
  <groupId>org.apache.kafka</groupId>
  <artifactId>kafka-clients</artifactId>
  <version>2.6.0</version>
</dependency>
```

Create Kafka Producer to send records

Tos

- ❖ Specify bootstrap servers
- ❖ Specify client.id
- ❖ Specify Record Key serializer
- ❖ Specify Record Value serializer

Common Kafka imports and constants

```
KafkaProducerExample.java x
KafkaProducerExample
1 package com.cloudurable.kafka;
2
3 import org.apache.kafka.clients.producer.*;
4 import org.apache.kafka.common.serialization.LongSerializer;
5 import org.apache.kafka.common.serialization.StringSerializer;
6
7 import java.util.Properties;
8
9 public class KafkaProducerExample {
10
11     private final static String TOPIC = "my-example-topic";
12     private final static String BOOTSTRAP_SERVERS =
13         "localhost:9092,localhost:9093,localhost:9094";
14 }
```

Create Kafka Producer to send records

```
KafkaProducerExample.java x
KafkaProducerExample

14
15 private static Producer<Long, String> createProducer() {
16     Properties props = new Properties();
17     props.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
18               BOOTSTRAP_SERVERS);
19     props.put(ProducerConfig.CLIENT_ID_CONFIG, "KafkaExampleProducer");
20     props.put(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
21               LongSerializer.class.getName());
22     props.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
23               StringSerializer.class.getName());
24     return new KafkaProducer<>(props);
25 }
26
```

Send async records with Kafka Producer

```
KafkaProducerExample.java x
KafkaProducerExample runProducer()

27
28 static void runProducer(final int sendMessageCount) throws Exception {
29     final Producer<Long, String> producer = createProducer();
30     long time = System.currentTimeMillis();
31
32     try {
33         for (long index = time; index < time + sendMessageCount; index++) {
34             final ProducerRecord<Long, String> record =
35                 new ProducerRecord<>(TOPIC, index,
36                                     value: "Hello Mom " + index);
37
38             RecordMetadata metadata = producer.send(record).get();
39
40             long elapsedTime = System.currentTimeMillis() - time;
41             System.out.printf("sent record(key=%s value=%s) " +
42                             "meta(partition=%d, offset=%d) time=%d\n",
43                             record.key(), record.value(), metadata.partition(),
44                             metadata.offset(), elapsedTime);
45         }
46     } finally {
47         producer.flush();
48         producer.close();
49     }
50 }
51 }
```

```
public static void main(String... args) throws Exception {  
    if (args.length == 0) {  
        runProducer( sendMessageCount: 5);  
    } else {  
        runProducer(Integer.parseInt(args[0]));  
    }  
}
```

KafkaProducer<K,V>

```
Properties props = new Properties();
props.put("bootstrap.servers", "localhost:9092");
props.put("acks", "all");
props.put("key.serializer", "org.apache.kafka.common.serialization.StringSerializer");
props.put("value.serializer", "org.apache.kafka.common.serialization.StringSerializer");

Producer<String, String> producer = new KafkaProducer<>(props);
for (int i = 0; i < 100; i++)
    producer.send(new ProducerRecord<String, String>("my-topic", Integer.toString(i), Integer.toString(i)));

producer.close();
```

org.apache.kafka.clients.producer

Interface Callback

```
public interface Callback
```

A callback interface that the user can implement to allow code to execute when the request is complete. This callback will generally execute in the background I/O thread so it should be fast.

Method Summary

Methods

Modifier and Type	Method and Description
void	onCompletion (RecordMetadata metadata, Exception exception) A callback method the user can implement to provide asynchronous handling of request completion.

send

```
public Future<RecordMetadata> send(ProducerRecord<K,V> record,  
                                   Callback callback)
```

Asynchronously send a record to a topic and invoke the provided callback when the send has been acknowledged.

The send is asynchronous and this method will return immediately once the record has been stored in the buffer of records waiting to be sent. This allows sending many records in parallel without blocking to wait for the response after each one.

- ❖ Used to send a record to a topic
- ❖ provided callback gets called when the send is acknowledged
- ❖ Send is asynchronous, and method will return immediately
 - ❖ once the record gets stored in the buffer of records waiting to post to the Kafka broker
- ❖ Allows sending many records in parallel without blocking

```
String val = "Hello Kafka :" + Integer.toString(i);
producer.send(
    new ProducerRecord<String, String>("my-first-topic", Integer.toString(i), val)
    , (recordMetadata, exception) -> {
        if (exception == null) {
            System.out.println("Record written to offset " +
                recordMetadata.offset() + " timestamp " +
                recordMetadata.timestamp());
        } else {
            System.err.println("An error occurred");
            exception.printStackTrace(System.err);
        }
    });
```

Progress Console

<terminated> KafkaProducerExample (1) [Java Application] /Library/Java/JavaVirtualMachines/jdk1.8.0_271.jdk/Contents/Home/bin/java (14-Mar-2022, 3:38:16 PM -

SLF4J: Failed to load class "org.slf4j.impl.StaticLoggerBinder".
SLF4J: Defaulting to no-operation (NOP) logger implementation
SLF4J: See <http://www.slf4j.org/codes.html#StaticLoggerBinder> for further details.

Record written to offset 2 timestamp 1647252504793
Sent : 0 Record written to offset 3 timestamp 1647252504878
Sent : 1

- ❖ Two forms of send with callback and with no callback both return Future
 - ❖ Asynchronously sends a record to a topic
 - ❖ Callback gets invoked when send has been acknowledged.
- ❖ send is asynchronous and return right away as soon as record has added to send buffer
- ❖ Sending many records at once without blocking for response from Kafka broker
- ❖ Result of send is a RecordMetadata
 - ❖ record partition, record offset, record timestamp
- ❖ Callbacks for records sent to same partition are executed in order

- Verify that replication is working with kafka-replica-verification
- Utility that ships with Kafka, If lag or outage you will see it as follows:

```
$ kafka/bin/kafka-replica-verification.sh --broker-list localhost:9092 --topic-white-list my-example-topic
2017-05-17 14:06:46,446: verification process is started.
2017-05-17 14:07:16,416: max lag is 0 for partition [my-example-topic,12] at offset 197 among 13 partitions
2017-05-17 14:07:46,417: max lag is 0 for partition [my-example-topic,12] at offset 201 among 13 partitions

2017-05-17 14:36:47,497: max lag is 11 for partition [my-example-topic,5] at offset 272 among 13 partitions
2017-05-17 14:37:19,408: max lag is 15 for partition [my-example-topic,5] at offset 272 among 13 partitions
...
2017-05-17 14:38:49,607: max lag is 0 for partition [my-example-topic,12] at offset 272 among 13 partitions
```

```
[root@kafka0 kafka-logs]# kafka-replica-verification.sh --broker-list kafka0:9092 --topic-white-list ratings
2022-03-14 04:26:16,536: verification process is started.
2022-03-14 04:26:46,748: max lag is 0 for partition ratings-0 at offset 20 among 1 partitions
2022-03-14 04:27:16,880: max lag is 0 for partition ratings-0 at offset 20 among 1 partitions
2022-03-14 04:27:47,072: max lag is 0 for partition ratings-0 at offset 20 among 1 partitions
2022-03-14 04:28:17,204: max lag is 0 for partition ratings-0 at offset 20 among 1 partitions
2022-03-14 04:28:47,326: max lag is 0 for partition ratings-0 at offset 20 among 1 partitions
```

Custom Partitioner Architecture

The Partitioner Interface

Implements custom routing logic for specific message keys.

Control Destination

Directs targeted payloads to specific physical partitions.

Key Routing Rules

Overrides default hashing algorithm via custom business rules.

Custom Partitioner Java Example

Routing Implementation Steps

1. **Contract:** Implement *Partitioner* interface.
2. **Logic:** Intercept key and return target partition index.
3. **Config:** Register class via *partitioner.class*.

```
public class VIPPartitioner implements Partitioner {  
    public int partition(String topic, Object key,  
        byte[] keyBytes, Object value, byte[] valBytes,  
        Cluster cluster) {  
        List<PartitionInfo> partitions =  
            cluster.partitionsForTopic(topic);  
        int numPartitions = partitions.size();  
        if (((String) key).equals("VIP_USER")) {  
            return 0; // Route VIP keys to partition 0  
        }  
        return Math.abs(Utils.murmur2(keyBytes))  
            % (numPartitions - 1) + 1;  
    }  
    public void configure(Map configs) {}  
    public void close() {}  
}
```

Lab : Java API – Producer – JSON Object Serialization- 60 Minutes